



Node.js y Express

HTTP, JWT, CRUD, middleware y comunicación entre archivos

HTTP

GET /productos

Comunicación entre cliente y servidor

JWT

`header.payload.firma`

Autenticación mediante tokens

CRUD

C R U D

Operaciones básicas de un backend

PETICIONES HTTP

AUTENTICACIÓN

BACKEND

ESTUDIANTE	BOOTCAMP	FECHA
Juan Andrés Suárez Gaviria	Desarrollo web fullstack	9 de septiembre de 2026

Actividad de investigación | Fundamentos de backend con Node.js

Peticiones HTTP

DEFINICIÓN

Una petición HTTP es un mensaje que un cliente envía a un servidor para consultar información o ejecutar una acción. El servidor procesa la solicitud y devuelve una respuesta con un estado, encabezados y, normalmente, datos en JSON.

¿Para qué sirven?

Permiten conectar navegadores, aplicaciones móviles y otros sistemas con un backend. Gracias a ellas se pueden iniciar sesiones, consultar productos, registrar usuarios, actualizar información o eliminar recursos.

Ciclo de petición y respuesta

1 Cliente

Realiza una acción

2 Petición

Método, URL y datos

3 Servidor

Node y Express procesan

4 Respuesta

Estado y contenido JSON

Anatomía de una petición

EJEMPLO HTTP

```
POST /productos HTTP/1.1
Content-Type: application/json
Authorization: Bearer token

{ "nombre": "Teclado", "precio": 120000 }
```

Métodos y códigos frecuentes

Método	Uso
GET	Consultar recursos
POST	Crear un recurso
PUT / PATCH	Actualizar información
DELETE	Eliminar un recurso

Estado	Significado
200	Solicitud correcta
201	Recurso creado
400	Datos incorrectos
401	No autenticado
404	No encontrado
500	Error del servidor

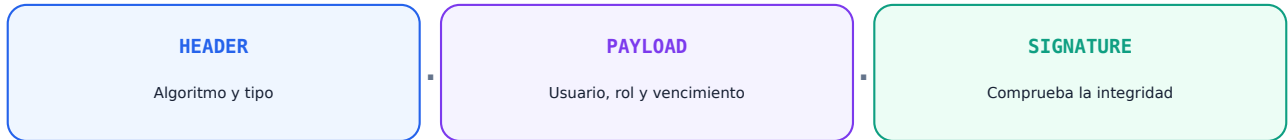
HTTP no recuerda automáticamente peticiones anteriores. La identidad o el estado se mantienen mediante mecanismos como sesiones, cookies o tokens.

JWT y su uso en Express

DEFINICIÓN

JWT significa JSON Web Token. Es un formato compacto para transmitir datos verificables entre dos partes. En una API suele representar la identidad y los permisos de un usuario después de iniciar sesión.

Estructura de un JWT firmado



El payload se puede leer: no debe contener contraseñas ni información secreta. La firma evita modificaciones, pero no cifra automáticamente el contenido.

Flujo de autenticación



Creación y verificación

CREAR TOKEN

```
const jwt = require("jsonwebtoken");

const token = jwt.sign(
  { usuario: "admin", rol: "admin" },
  process.env.JWT_SECRET,
  { expiresIn: "1h", algorithm: "HS256" }
);
```

VERIFICAR TOKEN

```
try {
  const datos = jwt.verify(
    token,
    process.env.JWT_SECRET,
    { algorithms: ["HS256"] }
  );
  req.usuario = datos;
  next();
} catch (error) {
  res.status(401).json({ mensaje: "Token inválido" });
}
```

Buena práctica	Aplicación
Secreto fuera del código	Guardar JWT_SECRET en un archivo .env
Vencimiento	Usar expiresIn para limitar la duración
Verificación	Usar jwt.verify(), no solamente jwt.decode()

CRUD en un backend

DEFINICIÓN

CRUD reúne las cuatro operaciones básicas para administrar información: crear, consultar, actualizar y eliminar. En una API estas acciones se relacionan normalmente con métodos HTTP.

C**CREATE**

POST

R**READ**

GET

U**UPDATE**

PUT / PATCH

D**DELETE**

DELETE

Rutas del CRUD

Método	Ruta	Acción	Respuesta esperada
GET	/productos	Listar todos	200 OK
GET	/productos/:id	Consultar uno	200 o 404
POST	/productos	Crear	201 Created
PUT	/productos/:id	Actualizar	200 o 404
DELETE	/productos/:id	Eliminar	200 o 404

Ejemplo de controlador

LEER Y CREAR

```
let productos = [];  
  
function listar(req, res) {  
  res.json(productos);  
}  
  
function crear(req, res) {  
  const nuevo = { id: Date.now(), ...req.body };  
  productos.push(nuevo);  
  res.status(201).json(nuevo);  
}
```

ACTUALIZAR Y ELIMINAR

```
function actualizar(req, res) {  
  const producto = productos.find(  
    item => item.id === Number(req.params.id)  
  );  
  if (!producto) return res.sendStatus(404);  
  Object.assign(producto, req.body);  
  res.json(producto);  
}  
  
function eliminar(req, res) {  
  productos = productos.filter(  
    item => item.id !== Number(req.params.id)  
  );  
  res.json({ mensaje: "Producto eliminado" });  
}
```

Este ejemplo guarda los datos en memoria. Al reiniciar el servidor se eliminan; una aplicación real utilizaría una base de datos.

Middleware en Express

DEFINICIÓN

Un middleware es una función que participa en el ciclo de una petición. Puede ejecutar código, leer o modificar req y res, terminar la respuesta o entregar el control al siguiente paso mediante next().

Orden de ejecución

1 Petición

Llega al servidor

2 Registro

Guarda método y URL

3 JWT

Verifica el acceso

4 Controlador

Ejecuta la operación

5 Respuesta

Devuelve JSON

Tipos y ejemplos

Tipo	Ejemplo	Función
Incorporado	express.json()	Convierte el cuerpo JSON en req.body
Aplicación	app.use(registro)	Se ejecuta para todas las peticiones
Ruta	router.post('/', validar)	Se ejecuta en una ruta específica
Errores	(err, req, res, next)	Centraliza respuestas de error

Ejemplos de middleware

REGISTRO

```
function registrar(req, res, next) {
  console.log(
    `${req.method} ${req.originalUrl}`
  );
  next();
}

app.use(registrar);
```

AUTENTICACIÓN

```
function verificarToken(req, res, next) {
  const auth = req.headers.authorization;

  if (!auth?.startsWith("Bearer ")) {
    return res.status(401).json({
      mensaje: "Token requerido"
    });
  }

  next();
}
```

Si un middleware no envía una respuesta, debe llamar a next(). De lo contrario, la petición queda esperando indefinidamente.

Comunicación entre archivos

MÓDULOS

Node.js permite dividir el proyecto en archivos especializados. Un archivo exporta funciones u objetos y otro los importa para utilizarlos. Esto mejora el orden, la reutilización y el mantenimiento.

Organización del proyecto

ESTRUCTURA

```
crud-productos/  
|-- app.js  
|-- controllers/  
|   |-- productos.controller.js  
|-- middlewares/  
|   |-- auth.middleware.js  
|-- routes/  
|   |-- productos.routes.js
```

1 app.js

Configura Express

2 Rutas

Define endpoints

3 Controlador

Procesa datos

app.js importa las rutas; las rutas importan los controladores. Cada archivo cumple una responsabilidad concreta.

CommonJS: exportar e importar

ARCHIVO QUE EXPORTA

```
// operaciones.js  
function sumar(a, b) {  
  return a + b;  
}  
  
module.exports = { sumar };
```

ARCHIVO QUE IMPORTA

```
// app.js  
const { sumar } = require("./operaciones");  
  
console.log(sumar(5, 3));  
// Resultado: 8
```

Dos sistemas disponibles

Sistema	Exportar	Importar	Configuración
CommonJS	module.exports	require()	Común en proyectos Node tradicionales
ES Modules	export	import	Usa type: module o archivos .mjs

En rutas relativas se utiliza ./ para la carpeta actual y ../ para subir un nivel. Por ejemplo: require('../controllers/productos.controller').

Mini backend completo

Instalación: `npm init -y | npm install express jsonwebtoken dotenv` | Ejecución: `node app.js`

Archivo principal: app.js

APP.JS

```
require("dotenv").config();
const express = require("express");
const jwt = require("jsonwebtoken");
const rutas = require("../routes/productos.routes");
const verificar = require("../middlewares/auth.middleware");

const app = express();
app.use(express.json());

app.post("/login", (req, res) => {
  const { usuario, clave } = req.body;
  if (usuario !== "admin" || clave !== "1234") {
    return res.status(401).json({ mensaje: "Datos incorrectos" });
  }

  const token = jwt.sign(
    { usuario, rol: "administrador" },
    process.env.JWT_SECRET,
    { expiresIn: "1h", algorithm: "HS256" }
  );
  res.json({ token });
});

app.use("/productos", verificar, rutas);
app.listen(3000, () => console.log("Servidor en puerto 3000"));
```

AUTH.MIDDLEWARE.JS

```
const jwt = require("jsonwebtoken");

function verificar(req, res, next) {
  const auth = req.headers.authorization;
  if (!auth?.startsWith("Bearer ")) {
    return res.sendStatus(401);
  }

  try {
    req.usuario = jwt.verify(
      auth.split(" ")[1],
      process.env.JWT_SECRET,
      { algorithms: ["HS256"] }
    );
    next();
  } catch (error) {
    res.sendStatus(401);
  }
}

module.exports = verificar;
```

Archivo de rutas

PRODUCTOS.ROUTES.JS

```
const router = require("express").Router();
const controller = require("../controllers/productos.controller");

router.get("/", controller.listar);
router.get("/:id", controller.obtenerPorId);
router.post("/", controller.crear);
router.put("/:id", controller.actualizar);
router.delete("/:id", controller.eliminar);

module.exports = router;
```

El controlador contiene las funciones del CRUD mostradas en la página 4. La ruta recibe la petición y ejecuta la función correspondiente.

Prueba del backend

HERRAMIENTAS

El backend puede probarse con Postman, Insomnia, Thunder Client o curl. Primero se inicia sesión para obtener el JWT y después se utiliza ese token en las rutas protegidas.

1. Obtener el token

PETICIÓN

```
POST http://localhost:3000/login
Content-Type: application/json

{
  "usuario": "admin",
  "clave": "1234"
}
```

RESPUESTA

```
HTTP/1.1 200 OK

{
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

2. Enviar el JWT

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

3. Probar las operaciones CRUD

Prueba	Método y ruta	Cuerpo JSON	Resultado
Listar	GET /productos	No requiere	Arreglo de productos
Crear	POST /productos	nombre y precio	Nuevo producto
Consultar	GET /productos/1	No requiere	Producto o 404
Actualizar	PUT /productos/1	datos modificados	Producto actualizado
Eliminar	DELETE /productos/1	No requiere	Confirmación

Recorrido completo

1 Cliente

Envía HTTP + JWT

2 Middleware

Valida el token

3 Ruta

Elige la operación

4 Controlador

Aplica el CRUD

5 JSON

Devuelve resultado

Comprobación	Resultado esperado
Sin token	401 - acceso rechazado
Token válido	La ruta procesa la petición
ID inexistente	404 - recurso no encontrado
Creación correcta	201 - recurso creado